Program-3

Neural Network for News Wires Classification

Aim: Design a neural Network for classifying news wires (Multi class classification) using Reuters dataset.

Introduction to IMDB Dataset:

The Reuters Newswire Dataset is the standard benchmark for Multi-Class Text Classification.

- **Input** : A short news article published by Reuters in 1986.
- **Output (Classes)** : One of 46 different topics (e.g., "Earnings," "Coffee," "Acquisitions," "Grain," "Gold," "Ship," etc.).
- **Total Samples** : 11,228 distinct newswires.
  - Training : 8,982
  - Testing : 2,246
- **Reuters Dataset is unbalanced**:
  - Some topics are huge, and some are tiny.
  - The topic "earn" (earnings) appears thousands of times.
  - The topic "vegetable oil" might only appear 5 times.

  This makes it harder for the model to learn the rare topics, because it sees them so frequently.

Note:

i. Neural networks cannot handle raw lists of integers of varying lengths. We must convert them into fixed-size tensors.

ii. The function (***Vectorize***) creates a matrix where every row is a review and every column is a specific word (from 0 to 9,999).

iii. Multi-Hot Encoding: If the word "apple" corresponds to index 5 and appears in the article, res[i, 5] becomes 1.0.

iv. Result: A "Bag of Words" representation. The model knows which words are present, but not their order.

Modeling the network:

1. Input layer : input is equal to number of words
2. Hidden layer-1 : neurons = 64, activation function = ‘relu’
3. Hidden layer-2 : neurons = 64, activation function = ‘relu’
4. Output layer : neurons = 46, activation function = ‘softmax’
5. Loss function : categorical cross entropy
6. Optimization function : adam
7. Metrics : accuracy

Modules used:

1. Tensorflow, Keras : for modeling the network
2. Matplotlib : for graphical visualization of result

Program:

```
#Import the required modules
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import reuters
from tensorflow.keras.utils import to_categorical
import numpy as np
import matplotlib.pyplot as plt
#Import the dataset
num_words = 10000
(x_train, y_train), (x_test, y_test)=reuters.load_data(num_words=num_words)
#Multi-hot-encoding of words
def vectorize(seqs, dim=num_words):
    res=np.zeros((len(seqs), dim), dtype="float32")
    for i, seq in enumerate(seqs):
        res[i, seq] = 1.0
    return res
x_train = vectorize(x_train)
x_test = vectorize(x_test)
#One-hot-encoding of words
y_train_oh = to_categorical(y_train)
y_test_oh = to_categorical(y_test)
#Model the network
model = keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(num_words,)),
    layers.Dense(64, activation='relu'),
    layers.Dense(46, activation='softmax')
])
model.compile(optimizer='adam',
            loss='categorical_crossentropy',metrics=['accuracy'])
#Train and evaluate the model
```

```
history=model.fit(x_train, y_train_oh, epochs=10, batch_size=512, validation_split=0.15)
test_loss, test_acc=model.evaluate(x_test, y_test_oh)
print(f"Test accuracy:{test_acc}")
print(f"Test loss:{test_loss}")
#Plot the graph
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.savefig('Model Accuracy.jpg')
plt.show()
```

Output: Test accuracy: 0.8673

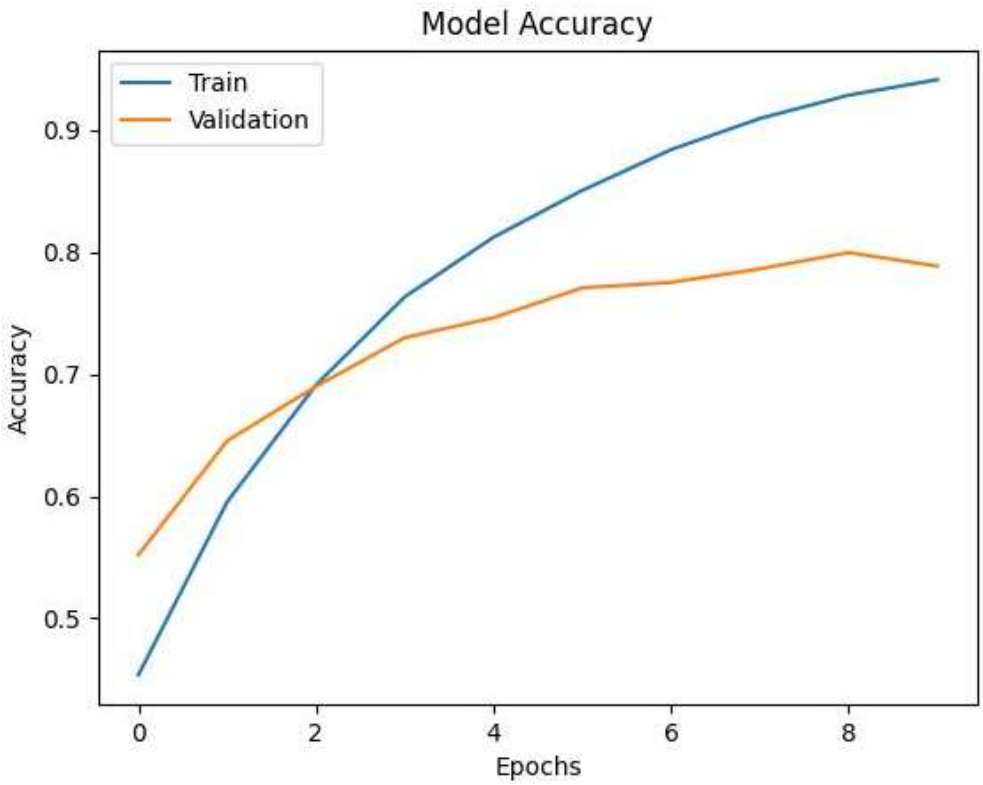

Fig. 1 Model Accuracy plot